

Reducing the Cost of Regression Testing by Semantics Guided Test Case Selection

David Binkley[†]

Computer Science Department
Loyola College in Maryland
4501 North Charles Street
Baltimore, Maryland 21210
binkley@cs.loyola.edu

Abstract

Software maintainers are faced with the task of regression testing: retesting a modified program on a (large) number of test cases. The cost of regression testing can be reduced if old test cases and old test results can be reused. Reuse avoids the costly construction of new test cases and the unproductive rerunning of existing test cases when it can be guaranteed that the modified and original programs will produce the same results. An algorithm that uses language semantics to provide such a guarantee is presented. This algorithm uses semantic (not syntactic) differences and similarities between the old and new programs.

The algorithm is based on the notion of common execution patterns, which is the interprocedural extension of equivalent execution patterns. Program components with common execution patterns are computed using a new type of interprocedural slice called a calling context slice. Whereas an interprocedural slice includes the program components necessary to capture all possible executions of a statement, a calling context slice includes only those program components necessary to capture the execution of a statement in a particular calling context (i.e., a particular call to the procedure).

CR Categories and Subject Descriptors: D.2.2 [Software Engineering]: Tools and Techniques—programmers workbench; D.2.5 [Software Engineering]: Testing and Debugging; D.3.3 [Programming Languages]: Language Constructs—control structures, procedures, functions, and subroutines; E.1 [Data Structures] graphs

Other Keywords: regression testing, slicing.

[†]supported in part by a grant from the David S. Lattanze Center for Executive Studies and National Science Foundation grant CCR-9411861

1. Introduction

Software maintainers are faced with the task of regression testing: the process of retesting software after a modification. This process may involve running the modified program on a large number of test cases, even after the smallest of changes (the top three entries of Weinberg's list of the largest software disasters are one line changes that were not tested [22]). Although the effort required to make a small change may be minimal, the effort required to recertify the modified program after such a change may be substantial.

This paper reports our recent work on reducing the cost of regression testing. Given a program, certified, which passes some test suite, and a modified version of this program, modified, we present an algorithm that reuses test cases and test results from certified's testing. Certified's test cases are assumed to provide adequate coverage [1, 7, 12, 14, 18]. Our algorithm partitions these test cases into those that produce the same output in certified and modified and those that may produce different results. It also identifies components of modified that cannot be guaranteed to be covered by any of certified's tests. Reusing test cases avoids the costly construction of new test cases. Reusing test results avoids the expense of running modified on test cases for which it can be guaranteed that modified and certified will produce the same results.

Recent trends in software engineering encourage the use of many small procedures (e.g., OOP), which makes it essential that algorithms for reducing the cost of regression testing handle the interprocedural impact of changes. To identify reusable test cases and test results in a program with procedures and procedure calls, we introduce the notion of *common execution patterns*. An interprocedural extension of equivalent execution patterns [1], common execution patterns capture the *semantic* (not syntactic) differences and similarities between program components.

To identify statements that have common execution patterns a new kind of interprocedural slice, called a *calling context slice*, is introduced. This slice captures the behavior of a statement on a particular invocation of its procedure. Thus, it contains fewer program components than an *interprocedural slice*, but more than an *intraprocedural slice*. One advantage of this kind of slice is that it allows the algorithm to better handle multiple changes to certified. Previous approaches, for example [10,16], make the assumption that a test case exercises the same components in certified and modified. However, a change in one computation might affect the path taken by a test case, which would invalidate this assumption.

The remainder of this paper is organized as follows: Section 2 presents background material. Section 3 describes how to partition the components of modified to facilitate test case selection. Section 4 presents our main technical contributions: it defines *common execution behavior*, *calling-context slice*, and presents algorithms for computing calling-context slices and for performing test case selection. Section 5 discusses related work and, finally, Section 6 contains a summary and discusses the present status of our implementation.

2. Background

This section provides background material on the system dependence graph (the intermediate representation used by our algorithms) and interprocedural slicing [11], the building of the program differences [2], and the use of test data adequacy criterion for testing software [1,23].

2.1. The System Dependence Graph and Interprocedural Slicing

The system dependence graph (SDG) represents a flat ("C"-like) language with recursive procedures where parameters are passed by value-result (other parameter passing mechanisms are also supported [3,11], but are not described due to space limitations). We assume programs contain no calls to non-existent procedures and that calls of the form $P(g)$ where g is a global variable have been rewritten by a preprocessing step to make g an explicit additional parameter of the procedure and the procedure call; thus, we do not discuss global variables explicitly. Output and input are modeled in the SDG as streams: the statement " $print(stream, x)$ " is treated as an assignment " $stream = append(stream, x)$." The use of streams connects all print statements using the same *stream* (transitively) with data dependence edges. At the same time output to different streams (files) do not effect each other (are not connected). Symmetrically, reading from an input stream removes a data value from the input stream, which modifies the stream.

An SDG is a collection of procedure dependence graphs (PDGs) connected by interprocedural control- and flow-dependence edges. Each PDG contains vertices that represent the components (statements and predicates) of the procedure and edges that represent the control and data-flow dependences between components. In addition, at call sites there are transitive dependence edges that summarize transitive dependences induced by called procedures. The dependences in a PDG are safe approximations to the semantic dependences found in the program [17], which are in general not computable and therefore must be approximated.

A slice of SDG G , taken with respect to a set of vertices S , contains those vertices of G whose components potentially affect the components represented in S . A slice is computed using two passes over G . Pass 1 starts from all vertices in S and goes backwards (from target to source) along the edges of the SDG without descending into called procedures. Pass 2 starts from all vertices reached in Pass 1 and goes backwards along the edges of the SDG but does not ascend to calling procedures. The result of an interprocedural slice is the set of vertices encountered during Pass 1 and Pass 2. The remainder of this paper uses the operator $b2$ to designate a Pass 2 slice.

Example. Figure 1 shows a program and part of its SDG. The slice of this SDG taken with respect to y as it is returned from procedure A includes the second call on procedure Add and the body of procedure Add . The two-step algorithm correctly omits the first call on procedure Add . (The stream "standard-out" is assumed in all the examples.)

The significance of a slice is that it captures a portion of the program's *behavior*. It is important to understand what is meant by program behavior. In the absence of procedures, behavior can be defined as the sequence of values produced by each component in the program. For example, an assignment statement produces the sequence of values assigned to the left-hand side and a conditional statement produces a sequence of boolean values. In the presence of procedures, this definition is too coarse. Instead, the behavior of a statement is the collection of sequences of values produced by a statement in each of the *calling-contexts* for its procedure. A calling context (formalized in Section 4.1) is the set of open procedures when a procedure is called.

2.2. Building the Program differences

The algorithm described in [2] uses interprocedural slices to construct the following three operators. These operators require being able to map a statement of modified to the corresponding statement certified. In practice such a map could be defined by applying a syntactic mapping matching algorithm [24], or by using a special editor that keeps track of statement correspondence as the program is modified. Such editors can be created by

```

procedure Main
  sum := 0
  i := 1
  while i ≤ 10 do
    call A(sum, i)
  od
  print(sum)
end

```

```

procedure A(x, y)
  call Add(x, y)
  call Add(y, 1)
  return

```

```

procedure Add(a, b)
  a := a + b
  return

```

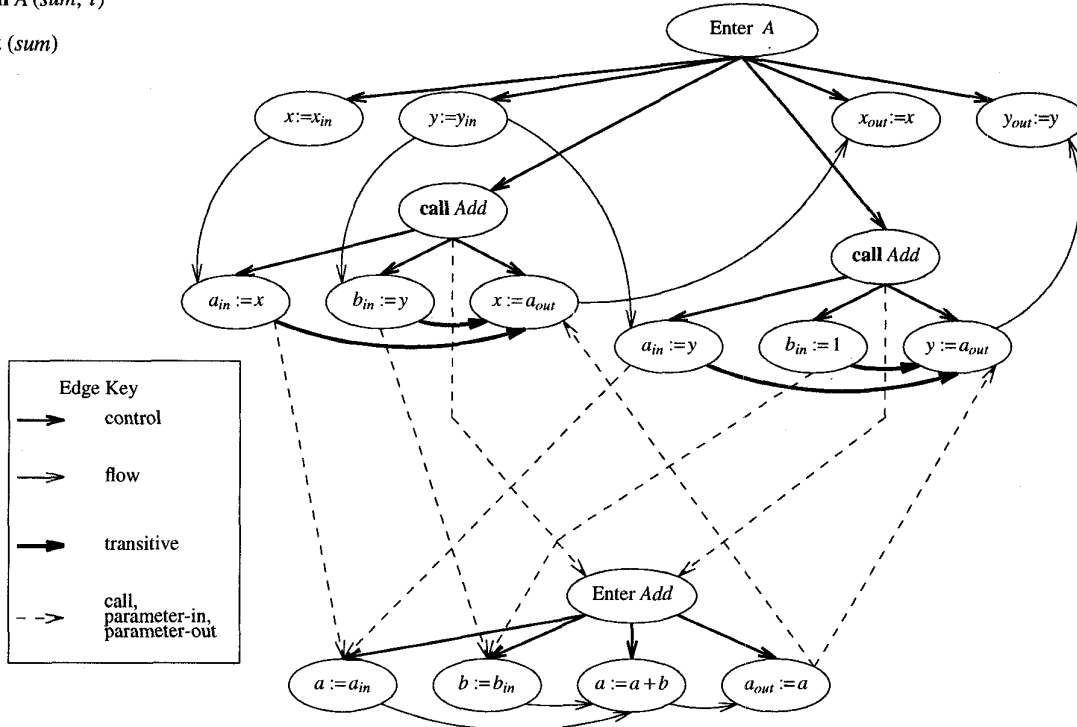


Figure 1. An example system that sums the numbers 1 to 10 and part of its SDG (only the PDGs for A and Add are shown).

MENTOR [8], GANDALF [15], and the Synthesizer Generator [19].

DAP(modified, certified)

The set of *directly affected points* contains the components of modified not in certified and those that have different incoming control and flow edges in modified and certified. This set is a safe approximation to the set of components from modified directly influenced by the change to certified.

AP(modified, certified)

The set of *affected points* contains the components of modified whose slice contains a directly affected point. This set is a safe approximation to the vertices of modified that represent components whose behavior differs (*i.e.*, whose interpretations differ as defined in [17]).

differences(modified, certified)

The program *differences* is an executable program obtained by taking slices with respect to affected points. It includes only those components of modified necessary to capture the *behavior* of modified that is different from the behavior of certified. Thus *differences*' *behavior* is identical to modified's when attention is restricted to affected points.

Program differences can be combined with the algorithm in Section 4 to further reduce the cost of regression testing: rather than retesting (the large) program modified using a (large) number of test cases, it is possible to certify that modified is adequately tested by running a (smaller) number of test cases on the (smaller) program differences.

2.3. Test Data Adequacy Criteria

A test data adequacy criterion is a minimum standard that a test suite (a set of test cases) for a program must satisfy. An example is the *all-statements* criterion, which requires that all statements in a program must be executed by at least one test case in the test suite. Satisfying an adequacy criterion provides some confidence that the test suite does a reasonable job of testing the program.

Test suite adequacy criteria can be divided into three groups: control-flow based criteria (e.g., all-statements), data-flow based criteria [23], and program dependence graph based criteria [1]. While the techniques discussed in Sections 3 and 4 are applicable to any of these, they work more naturally with criteria from the dependence graph group. We therefore consider the *all-vertices* and *all-flow-edges* criteria in Sections 3 and 4 as representative examples. These two are introduced below, but first, to relate them to the other groups, consider the following relationships from [1]:

- (1) the program dependence graph criterion all-vertices is equivalent to the control-flow criterion all-statements, and
- (2) the all-flow-edges criterion subsumes the data-flow criterion all-c-uses/some-p-uses (c-uses are computational uses and p-uses are predicate uses).

DEFINITION. (All-Vertices Criterion). The *all-vertices* criterion is satisfied by a set of test cases T if for each vertex v there is some test case t in T that exercises the statement represented by v . A statement (vertex) is *exercised* by test case t if the statement (statement corresponding to the vertex) is executed when the program is run with input t .

DEFINITION. (All-Flow-Edges Criterion). The *all-flow-edges* criterion is satisfied by a set of test cases T if for each flow edge e there is some test case t in T that exercises e . The flow edge $e = u \rightarrow_f v$, where u represents an assignment to variable x , is *exercised* if u is exercised; then no definition of x that lies on the control flow graph path taken from u to v is exercised, and finally v is exercised.

3. Partitioning of modified

This section discusses the partitioning of modified's components. The meaning of "components" varies depending on the testing criteria. For the two criteria considered below (all-vertices and all-flow edges), a component is a vertex or a flow-edge, respectively. Section 4 considers the impact of this partitioning on test case selection.

Identifying the components of modified that must be retested involves partitioning modified's components into the following sets: affected, deleted, new, and preserved. The new and deleted components are easily computed given the mapping between the statements of certified and modified. The bulk of the work in the computation of the set affected is done in the computation of the set of affected points AP . Finally, the preserved components are those components that are not in affected, deleted, or new. For the all-vertices and all-flow-edges these sets are defined in Table 1.

Example. Figure 2 illustrates these four sets for the all-flow-edges criterion.

4. Selection

This section introduces *common execution patterns* and discusses how they are used to perform test case selection. It then discusses calling-context slices, which are used to identify program components having common execution patterns. Finally, it states the complete algorithm for determining which test must be rerun and which components of modified must have new test cases devised for them.

4.1. Common Execution Patterns

Common execution patterns extend *equivalent execution patterns* [1] to programs that contain procedures and procedure calls. As their names indicate, *equivalent* is too strong in the presence of procedures because a component may be used in multiple contexts. Using equivalent execution patterns, while safe, is too coarse to be useful in the presence of procedures and procedure

Set	All Vertices	All Flow Edges
affected	$AP - \text{new}$	$\{ u \rightarrow_f v \mid v \in AP \}$
new	$\text{VertexSet}(\text{modified}) - \text{VertexSet}(\text{certified})$	$\text{FlowEdges}(\text{modified}) - \text{FlowEdges}(\text{certified})$
deleted	$\text{VertexSet}(\text{certified}) - \text{VertexSet}(\text{modified})$	$\text{FlowEdges}(\text{certified}) - \text{FlowEdges}(\text{modified})$
preserved	remaining vertices	remaining flow edges

Table 1

Certified	Modified	Identified Sets
[1] read(a)	read(a)	affected = { [1] $\rightarrow_f$ [4], [1] $\rightarrow_f$ [5], [2] $\rightarrow_f$ [4], [2] $\rightarrow_f$ [5], [4] $\rightarrow_f$ [6], [5] $\rightarrow_f$ [6] }
[2] read(b)	read(b)	
[3] if (a < 0)	if (b < 0)	
[4] $x = 1 / \text{pow}(a, -b)$	$x = 1 / \text{pow}(a, -b)$	new = { [2] $\rightarrow_f$ [3] }
else	else	deleted = { [1] $\rightarrow_f$ [3] }
[5] $x = \text{pow}(a, b)$	$x = \text{pow}(a, b)$	preserved = $\emptyset$
fi	fi	
[6] print(x)	print(x)	

Figure 2. Program “Certified” has a bug in it: a rather than b appears in the *if* statement (the auxiliary function *pow* takes only positive powers). The only directly affected point of modified is the predicate of the *if* statement. The affected points of modified are all the vertices subordinate to the *if* statement (which are reachable from the *if* statement via control edges), and statement [6], which can be reached via a flow dependence edge from affected points [4] and [5]. Because their targets are affected points, all the flow dependence edges to the two calls on procedure *pow* are in affected.

calls. The definition of *common execution patterns* identifies two components that share an equivalent execution pattern in some *calling context*, defined as follows:

DEFINITION (Calling Context). The *calling context* for procedure P (or a component in procedure P) is the sequence of call-sites that correspond to activation records currently on the stack when a particular activation of P is begun.

The definition of common execution patterns makes the assumption that both programs terminate normally. A program may fail to terminate normally if it contains a non-terminating loop or if a fault occurs, such as division by zero. If one program fails to terminate normally, it may not get (or get back to) the component being tested; thus, that component would execute fewer times in the non-terminating program.

DEFINITION (Common Execution Pattern). Components c_1 of procedure P_1 and c_2 of procedure P_2 have *common execution patterns* if there exist calling contexts CC_1 from P_1 and CC_2 from P_2 in which c_1 and c_2 are exercised the same number of times.

Recall that a vertex is exercised when the corresponding statement is executed, and that a flow edge is exercised when the source and target of the edge are exercised and no intervening definition of the variable defined at the source is exercised between the source and the target.

4.2. Test Case Selection

As described in Section 3, the components of modified are partitioned into four sets: preserved, deleted, new, and affected. We now describe how test cases are selected for each.

Components of preserved and deleted

Test cases that only exercise preserved or deleted components need not be rerun (test cases that exercise only deleted components can be removed from the test suite). For a small change, most test cases should test preserved components. Not rerunning these test cases should significantly reduce the cost of regression testing.

Components of new

For new components, which by definition have no test case devised for them, it may still be possible to reuse certified’s test cases. This is possible when, for example, a new vertex in a while-loop is tested by a test case for another vertex in the loop. More formally, if for some calling context CC , a component $new \in new$ and an existing component c have common execution patterns, then new is exercised by any test case t which exercises c . If no such component c exists then a new test case must be found (certified’s test suite provides a place to start looking for such tests).

Components of affected

The mistake most often made when considering an affected component a is to assume that a test case t that tests a in certified continues to test a in modified. The problem with this is that one of the changes to certified may have altered the components exercised by t . For example, replacing “<” with “>” in “if $x > 0$ ” has the effect of changing the path through the *if* statement that a test case with $x = 1$ would take. To avoid this mistake, components are matched by common execution pattern to identify appropriate test cases. This is facilitated by the auxiliary function *exercises*:

DEFINITION (Exercises). $exercises(t)$ maps test case t to the set of components exercised when certified is run on t .

Similar to new components, identifying test cases for an affected component a begins by identifying test cases that exercise a component c of certified such that a and c have common execution patterns. Let T be the set of all test cases t for which $c \in exercises(t)$ and c and a have a common execution pattern.

All tests in T exercise a ; however, not all test cases in T must be rerun. Component a is an affected point because its computation is reached by the computation of a directly affected point (DAP). Thus, test cases for modified that execute a , but do not execute a DAP need not be rerun. Only test cases that exercise both a and a DAP are chosen by the algorithm; test cases that involve a but no DAP will behave the same in certified and modified.

Example. Consider two calls on a procedure. If an affected component c in the called procedure is created by a change at the first call-site, then tests involving (only) calls from the second call-site need not be rerun.

Example. Consider the programs shown in Figure 3. In this example the vertex representing statement [6] is in affected. Although both test cases exercise this vertex, only test t_1 causes a DAP to be executed. Test case t_2 is guaranteed to produce the same result in certified and modified and therefore does not need to be rerun.

Note that if the statement “**read(a)**” is replaced by “**a=2**” in Figure 3, then “**read(b)**” would be a directly affected point because of the stream model used for input. This not only illustrates the use of the stream model, but also demonstrates how the algorithm handles deletions from certified.

A further reduction in the number of test cases to be rerun is possible if, with each affected component, the set of the DAPs that cause the component to be in affected is kept. This set is used to weed out test cases that exercise c and a DAP that does *not* affect c . Such test cases produce the same results in modified and certified.

4.3. Calling-Context Slice

A *calling-context slice*, taken with respect to vertex v and calling context CC , includes those statements necessary to capture the computation of v in calling context CC , but no other calling contexts. Thus, a calling-context slice contains less of the program than an interprocedural slice, but more of the program than an intraprocedural slice. An algorithm that computes calling-context slices is stated in Figure 4. This algorithm uses $b2$ (second pass) interprocedural slices, which include the necessary statements in a procedure (and called procedures), but ignore calling procedures. By repeatedly taking $b2$ slices back through the call-sites that make up the calling-context, only those parts of the program that contribute to the execution of the statement represented by v in calling context CC are included.

This algorithm is used to identify components with common execution patterns. For vertices, it is sufficient to capture the complete execution *behavior* of the statement upon which a vertex is control dependent by taking the calling context slice with respect to this vertex. For a flow edge, it is necessary for the source, target, and all intervening definitions of the variable defined at the source to have equivalent execution patterns in the same common calling context. It is shown in [5] that two

	Certified	Modified	Certified's Test Cases
[1]	read(a)	read(a)	
[2]	read(b)	read(b)	
[3]	if ($b < 0$)	if ($b < 0$)	$t_1 = 2 \ -1$
[4]	$x = pow(a, -b)$ else	$x = 1 / pow(a, -b)$ else	(vertices tested) [1], [2], [3], [4], and [6]
[5]	$x = pow(a, b)$ fi	$x = pow(a, b)$ fi	$t_2 = 2 \ 4$
[6]	print (x)	print (x)	(vertices tested) [1], [2], [3], [5], and [6]

Figure 3. This example assumes all-vertices coverage. The “certified” program has a bug in it: when b is less than zero, the result of $pow(a, -b)$ should be inverted. The fix introduces DAP [4], and affected points [4] and [6]. While the execution of test case t_2 includes statement [6], this case need not be rerun because it tests path [1], [2], [3], [5], [6], which does not include a DAP.

```

function CallingContextSlice( $G, v, CC$ ) returns a set of vertices
declare
   $G$ : an SDG
   $v$ : a vertex of  $G$ 
   $CC$ : a calling context (a list of call-sites)
   $tmp, Answer$ : sets of vertices of  $G$ 
begin
   $Answer := \emptyset$ 
   $tmp := \{ v \}$ 
  while  $CC \neq \text{empty}$  do
     $Answer := Answer \cup b2(G, tmp)$ 
     $tmp := \{ a \mid a \text{ is an actual-in vertex at call-site } head(CC) \text{ whose corresponding formal-in vertex is in } b2(G, tmp) \}$ 
     $CC := tail(CC)$ 
  od
   $Answer := Answer \cup b2(G, tmp)$ 
  return( $Answer$ )
end

```

Figure 4. Function `CallingContextSlice` returns all vertices of G that affect the behavior of v when v 's procedure is called through the sequence of call-sites in calling context CC .

vertices whose control successors have isomorphic¹ calling context slices have common execution patterns. The same is true for two flow edges.

4.4. Complete Algorithm

The complete algorithm for test case selection is stated in Figure 5. The algorithm has two outputs: the subset of certified's test cases to be rerun and the components of modified not guaranteed to be covered by any of certified's test cases. Although the semantic properties of slices allow us to prove the algorithm's correctness, such a proof is omitted due to space limitations. It is possible to collect *all* test cases that test affected or new components of modified rather than just a sufficient subset. To do so, the inner if statement is replaced with

$$TestsToRerun = TestsToRerun \cup \{ t \mid c \in exercises(t) \text{ and } c \in C \}.$$

¹ Two slices are isomorphic if their *induced subgraphs* are isomorphic. The subgraph of graph G induced by vertices V contains V and the edges of G whose endpoints are in V . Two (induced sub)graphs G_1 and G_2 are isomorphic iff the following conditions are satisfied:

- (1) There is a 1-to-1 mapping g from the vertex set of G_1 onto the vertex set of G_2 and for every v in G_1 , v and $g(v)$ have the same text.
- (2) There is a 1-to-1 mapping h from the edge set of G_1 onto the edge set of G_2 and for every edge e in G_1 , e and $h(e)$ are of the same type (e.g., both control edges, or both flow edges, etc.) and have the same label.
- (3) For every edge $v \rightarrow u$ in G_1 , $h(v \rightarrow u) = g(v) \rightarrow g(u)$.

When G_1 and G_2 are isomorphic or when we are trying to prove G_1 and G_2 are isomorphic, for brevity, we will say v and $g(v)$ are the *same* vertex and e and $h(e)$ are the *same* edge.

5. Related Work

The solution presented in Section 3 can be viewed as a pessimistic solution to the test case selection problem because it does not assume that test cases exercise the same components in certified and modified. This is in contrast to the optimistic solutions considered, for example, in [16] and [10]. Optimistic solutions make the assumption that test case t will exercise the same components in certified and modified. Since a change in one part of certified may affect the flow of control through modified, this assumption may be invalid. To handle cases in which this optimism is misguided these techniques run the test cases they believe are required and record the actual components tested. New test cases are then devised to cover untested components. In comparison, the pessimistic approach deals with multiple changes to certified by partitioning components based on common execution patterns regardless of the number of changes made. In terms of computational costs, the optimistic approach avoids the cost of determining components with common execution patterns, but incurs the costs of running unnecessary test cases and the cost of creating and running new test cases. The cost of computing common execution patterns is bounded by the size of the program. The cost of running test cases is unbounded in the program's size; it is bounded only by the execution time of the program.

One advantage of the optimistic algorithm is when the pessimistic algorithm determines that a test case cannot be reused because it cannot safely show that some test tests the same components. For example, consider replacing `if  $a > 0$` with `if  $a \geq 0$` . This change may affect the direction taken by the if statement; thus, the

```

function TestCaseSelection(certified, modified, T) returns the set of tests to by rerun and the set of
                                                    components of modified that need new tests

declare
  certified, modified: the SDG's for programs certified and modified
  T: certified's test suite
  t, t': test cases
  c: a component of certified
  m: a component of modified
  TestsToRerun: the set of certified's test cases to be rerun
  ComponentsThatNeedTests: the set of modified's components not guaranteed to be tested by any of certified's test cases

begin
  TestsToRerun :=  $\emptyset$ 
  ComponentsThatNeedTests :=  $\emptyset$ 

  for each new or affected component of modified m do
    Let  $C = \{ c \in \text{certified} \mid c \text{ and } m \text{ have common execution patterns} \}$ 
    if  $C = \emptyset$  then
      Insert m into ComponentsThatNeedTests
    else
      if no  $c \in C$  exists such that  $t' \in \text{TestsToRerun}$  and  $c \in \text{exercises}(t')$  then
        Insert a test case  $t \in T$  such that  $c \in \text{exercises}(t)$  and  $t$  exercises a DAP into TestsToRerun
      fi
    fi
  od

  return(TestsToRerun, ComponentsThatNeedTests)
end

```

Figure 5. Function *TestCaseSelection* returns the set of *certified*'s test cases that must be rerun and the set of components from *modified* for which there are no test cases in *certified*'s test suite.

pessimistic approach cannot safely assume it takes the same path. But for a test with $a=10$, it does. Here the optimistic approach would discover this when it ran the $a=10$ test. The pessimistic approach may identify components in the if statement's body as needing new test cases. One place to search for new test cases is in *certified*'s test suite; for example, the test where $a=10$.

Rothermel and Harrold recently presented a regression test selection method that reduces the number of test cases [20]. They concentrate on the use of control dependence to identify cases that must be rerun. The algorithm does not directly incorporate the effects of changes in data dependence when identifying changes and their effects. To capture semantic changes rather than syntactic ones, and thus provide guarantees about the test cases selected, it is necessary to consider both control and data dependence changes.

An extension of their work [21], which accounts for both control and data dependence, first presents an intraprocedural test-case selection algorithm and then an interprocedural one. This new work makes several insightful observations about reducing the cost of retesting software. Both algorithms begin with the enter vertices of the (main) procedures of *certified* and *modified* and perform side-by-side post-order traversals of the two graphs. During the traversals, tests to be rerun are collected. The traversals stop at leaf nodes or

predicates that are directly affected points. To discover components that are not tested a second pass is made over *modified*'s graph. This pass uses slicing to identify untested components.

The extension to interprocedural test-case selection essentially applies the intraprocedural test-case selection algorithm to (a minor modification of) the SDG. This extension is less precise (selects a larger number of tests) than the approach presented herein because of approximations in certain cases involving interprocedural data dependences present in the modified SDG. The coverage algorithm for determining untested components requires a more substantial overhaul. The results are comparable to ours.

The two approaches have a very similar output, but contain radically different internals. While a thorough comparison would require stating Rothermel and Harrold's algorithm in detail, the following three differences are representative. First, the approach in this paper formalizes "exercised flow edge" better and consequently deals better with identifying new, affected, preserved, and deleted flow edges. This leads to better identification of which flow-edges need to be retested. For example, when an intervening definition (nested within a control predicate) is executed, the definition of *exercises* correctly captures only test cases that do not execute this intervening definition. Rothermel and Harrold's algorithm does not.

Second, for small changes we do less work because our algorithm starts with new and affected components, while Rothermel and Harrold's algorithm starts with the enter vertices of the two SDGs. It does this because it identifies the correspondence between certified and modified at the same time it computes the set of tests to be rerun. Pulling this identification out to a preprocessing step allows the identification to begin with new and affected components, but requires extra computation when only a small fraction of the correspondence is actually needed.

Finally, one contrast that can be made in the statement of the two algorithms is the use of higher-level operators (e.g., slicing operators) in the algorithm developed in this paper. This allows the hard problems of capturing semantic properties of programs using dependences and performing test-case selection to be separated. In contrast, the algorithms from [21] consider data and control dependences directly in the test case selection algorithm, which is in general more complicated and thus more error prone. It does, however, often produce more efficient algorithms.

With the exception of the technique described in this paper, and those in [9, 21], and [1], previous approaches identify changes syntactically. This accounts for direct changes to the program but may miss indirect ones. In contrast, capturing semantic change correctly handles directly affected and indirectly affected components.

For example, consider the following problem

Many debugging or updating fixes involve only simple changes One frequently made error is to change a variable assignment because the [right-hand-side expression] is incorrect, but not to check that the new [right-hand-side] expression is appropriate for all the [left-hand-side] variables' uses. If the results are used in several places, it is easy for the programmer to concentrate on only one or two of those uses, and not to be aware of the uses in other places [16].

Retesting of directly affected definition-use pairs (flow edges) is sufficient if the use is directly affected by the change. However, this change may affect a use "down stream" in the computation. The techniques presented in this paper correctly determine that these down-stream uses are also affected and must be retested.

6. Summary

We have presented an algorithm for reducing the cost of regression testing by reducing the number of test cases that must be created and run or rerun to ensure that a modified program is adequately tested. This algorithm is an important improvement over previous algorithms in one or more of three ways:

- (1) It uses semantic changes rather than syntactic changes to identify affected components of the

program and the test cases that must be rerun. To be clear, the semantics of a program are approximated by our techniques. Program semantics form the basis for the definitions of dependence, program slicing, common execution behavior, and differences. However, the actual algorithms, which are based on the semantic definitions, must necessarily work with programs. These algorithms capture safe approximations to the semantic definitions. For example, if the algorithm in Figure 4 determines that two components have isomorphic calling-context slices (a syntactic condition) then the components have common execution patterns (a semantic condition). However, if the algorithm cannot determine that two components have isomorphic calling-context slices, then it makes the safe assumption that they do not have common execution patterns. Erring on the safe side is necessary, since precise static analysis of semantic properties is an unsolvable problem.

- (2) On test cases that the algorithm determines do not need to be rerun, it *guarantees* that certified and modified will have the same behavior. A proof appears in [5]
- (3) It works in the presence of procedures and procedure calls.

We also defined the notion of common execution pattern, which is useful in understanding which test cases for certified test components of modified. The algorithm for calling-context slices provides a method of determining when two components have common execution patterns.

This test case selection algorithm, combined with the program differences allows a smaller number of test cases to be run on the smaller program differences. This approach works best on small and moderate changes of large programs where, as with the one line changes on Weinberg's list of the largest software disasters, high cost may make maintainers reluctant to perform the regression testing. Having a lower cost method that guarantees the same testing coverage could remove this reluctance.

We are in the process of constructing a prototype implementation for a subset of C. We are also exploring the possibility of implementing these algorithms using Unravel (a slicer for complete C programs) [13]. This would allow us to study the algorithms on larger "real-world" programs.

The prototype performs the following steps: SDG construction, slicing and constructing differences, and test-case selection. The complexity of these steps is as follows: The SDG construction is polynomial in the size of modified and certified [4]. Slicing and constructing differences are linear in the size of modified's SDG. And, finally, the algorithm in Figure 5 takes time proportional to the sum of the size of modified's SDG and the number of test cases in certified's test suite.

To achieve these bounds, the prototype implementation makes use of slightly different algorithms than presented in this paper. For example it employs forward slicing in addition to backwards slicing. These algorithms differ only in their efficiency. The system presently slices at approximately 1,000 lines (SDG vertices) per second on a DecStation 5000/133 without any performance tuning of the code. The partitioning algorithm and test case selection algorithm also run at essentially this speed. We are in the process of completing the implementation and devising appropriate tests to illustrate its effectiveness [6].

Acknowledgments

The limiting of tests to directly affected points was suggested to the author by Gregg Rothermel while the author was visiting Clemson University. Ed Birrane, Brad Kuhn, and the anonymous reviewers' comments improved and clarified the exposition.

References

1. Bates, S. and Horwitz, S., "Incremental program testing using program dependence graphs," in *Conference Record of the Twentieth ACM Symposium on Principles of Programming Languages*, (Charleston, SC, January 10-13, 1993), ACM, New York, NY (1993).
2. Binkley, D., "Using semantic differencing to reduce the cost of regression testing," pp. 41-50 in *Proceedings of the IEEE Conference on Software Maintenance*, (Orlando, FL, Nov, 1992), IEEE Computer Society, Washington, DC (1992).
3. Binkley, D., "Slicing in the presence of parameter aliasing," pp. 261-268 in *1993 Software Engineering Research Forum*, (Orlando, FL, Nov, 1993), (1993).
4. Binkley, D., Horwitz, S., and Reps, T., "Program Integration for Languages with Procedure Calls," *ACM Transactions on Software Engineering and Methodology*, (1995).
5. Binkley, D., "Interprocedural test case selection," Tech Rep. TR95001, Department of Computer Science, Loyola College, Baltimore, MD. (January 1995).
6. Binkley, D. and Birrane, E., "Experience with a new Regression Testing Algorithm," *submitted to the IEEE International Symposium on Software Testing and Analysis*, IEEE Computer Society, (1996).
7. Clarke, L.A., Podgurski, A., Richardson, D.J., and Zeil, S.J., "A formal evaluation of data flow path selection criteria," *IEEE Transactions on Software Engineering* **SE-15**(11) pp. 1318-1332 (November 1989).
8. Donzeau-Gouge, V., Huet, G., Kahn, G., and Lang, B., "Programming environments based on structured editors: The MENTOR experience," pp. 128-140 in *Interactive Programming Environments*, ed. D. Barstow, E. Sandewall, and H. Shrobe, McGraw-Hill, New York, NY (1984).
9. Gupta, R., Harrold, M.J., and Soffa, M.L., "An approach to regression testing using slicing," pp. 299-308 in *Proceedings of the IEEE Conference on Software Maintenance*, (Orlando, Florida, Nov, 1992), IEEE Computer Society, Washington, DC (1992).
10. Harrold, M.J. and Soffa, M.L., "An incremental approach to unit testing during maintenance," pp. 362-367 in *Proceedings of the IEEE Conference on Software Maintenance*, (Phoenix, Arizona, Oct, 1988), IEEE Computer Society, Washington, DC (1988).
11. Horwitz, S., Reps, T., and Binkley, D., "Interprocedural slicing using dependence graphs," *ACM Trans. Program. Lang. Syst.* **12**(1) pp. 26-60 (January 1990).
12. Howden, W.E., "Reliability of the path analysis testing strategy," *IEEE Transactions on Software Engineering* **SE-2**(3) pp. 208-215 (September 1976).
13. Lyle, J.R., Wallace, D.R., Graham, J.R., Gallagher, K.B., Poole, J.E., and Binkley, D.W., "A CASE tool to evaluate functional diversity in high integrity software," *In Preparation*. U.S. Department of Commerce, Technology Administration, National Institute of Standards and Technology, Computer Systems Laboratory, Gaithersburg, MD, (1995).
14. Marick, B., "A Survey of test effectiveness and cost studies," Report No. UIUCDCS-R-90-1652, Dept. of Computer Science, University of Illinois at Urbana-Champaign, Urbana, IL (December 1990).
15. Notkin, D., Ellison, R.J., Staudt, B.J., Kaiser, G.E., Kant, E., Habermann, A.N., Ambriola, V., and Montangero, C., Special issue on the GANDALF project, *Journal of Systems and Software* **5**(2)(May 1985).
16. Ostrand, T.J. and Weyuker, E.J., "Using data flow analysis for regression testing," in *Proceedings of the Sixth Annual Pacific Northwest Software Quality Conference*, (Portland, Oregon, September 19-20, 1988), ACM, New York, NY (1988).
17. Podgurski, A. and Clarke, L.A., "A formal model of program dependences and its implications for software testing, debugging, and maintenance," *IEEE Transactions on Software Engineering* **16**(9)(September 1990).
18. Rapps, S. and Weyuker, E.J., "Selecting software test data using data flow information," *IEEE Transactions on Software Engineering* **SE-11**(4) pp. 367-375 (1985).
19. Reps, T. and Teitelbaum, T., *The Synthesizer Generator: A system for constructing language-based editors*, Springer-Verlag, New York, NY (1988).
20. Rothermel, G. and Harrold, M.J., "A safe, efficient algorithm for regression test selection," *Proceedings of the IEEE Conference on Software Maintenance*, (Montreal, Quebec), pp. 358-367 IEEE Computer Society, (September 1993).
21. Rothermel, G. and Harrold, M.J., "Selecting tests and identifying test coverage requirements for modified software," *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis* Seattle, Washington, pp. 169-84 (August 1994).
22. Weinberg, G., "Kill that code!," *Infosystems* **48-49**(August 1983).
23. Weyuker, J., "The complexity of data flow criteria for test data selection," *Information Processing Letters* **19** pp. 103-109 (1984).
24. Yang, W., "Identifying syntactic differences between two programs," *Software—Practice and Experience* **21**(7)(July 1991).